통합반 11회차 — 낭독 대본

[이론 01] 현장 지식이 AI의 판단 근거가 되는 과정 복습하기

[SLIDE 01] 지식 · 에이전트 · 프로세스의 연결과 적용

안녕하세요. 이번 회차에서는 이 과정을 마무리합니다. 지금까지 쿨러 사건 하나를 붙잡고 세 영역을 나눠서 배웠죠. 판단의 재료가 되는 지식, 그 지식을 읽어서 추천안을 만드는 에이전트, 그리고 그 추천안을 사람의 승인과 시스템 실행으로 잇는 프로세스였어요.

사건을 한번 떠올려 볼까요? 쿨러에 이상이 생겨서 경보가 울렸고요, 에이전트가 원인을 가려서 조치를 추천했습니다. 운전원이 승인하자 시스템이 팬 속도를 올렸죠. 그런데 이 한 건 안에서 세 영역은 서로 무엇을 주고받았을까요? 그리고 이 구조를 쿨러가 아닌 내 업무에도 똑같이 세울 수 있을까요?

이번 회차에서는 먼저 쿨러 사건 한 건 위에서 세 영역이 주고받는 걸 되짚어 보고요, 그다음에 같은 구조를 각자의 작은 업무로 직접 만들어 봅니다. 마지막으로 실제 제품에서도 그 구조를 확인해 보겠습니다.

[SLIDE 02] 학습목표

이번 회차의 학습목표부터 볼게요. 화면에 카드가 네 장 있습니다.

첫 번째 카드는 지식에서 판단까지예요. 에이전트가 도구마다 어떤 지식과 어떤 값을 읽는지 설명하고, 검토와 개정을 거쳐 관리하는 지식과 지금의 값, 그리고 출처가 각각 어디에 있는지 가려 봅니다.

두 번째 카드는 제안에서 완료까지입니다. 추천안이 사람의 승인과 시스템 실행을 거쳐서 끝난 업무가 되는 길을 설명하고요, 에이전트 쪽 장치와 프로세스 쪽 장치가 각각 무엇을 맡는지 나눠 봐요.

세 번째 카드는 내 주제에 적용하기예요. 각자에게 친숙한 작은 업무 하나를 골라서 명세, 지식 구조, 에이전트, 흐름까지 직접 만듭니다. 이때 에이전트는 확인된 것만 제안하고, 나머지는 보류하게 할 거예요.

마지막 카드는 제품에서 확인하기입니다. ProcessGPT 시연을 보면서 배운 구조가 그대로 있는 자리와 넓어진 범위를 가려 보고, 끝으로 학습 노트에 개념과 산출물을 이어 둡니다.

[SLIDE 03] 목차

목차를 보면서 이번 회차의 순서를 잡아 볼게요. 화면에 여덟 줄이 있는데, 위의 다섯 줄이 이론입니다.

앞의 세 단원은 쿨러 사건의 경보 한 건이 추천안 한 건이 되기까지를 따라가요. 에이전트가 무엇을 받아서 시작하는지 보고, 그래프 질의와 증거로 원인을 가리는 과정, 그리고 조치를 정하고 출처를 남기는 방식을 차례로 봅니다.

넷째와 다섯째 단원은 그 추천안이 승인과 시스템 실행을 거쳐서 끝나기까지예요. 그 과정에 겹쳐 있는 에이전트 쪽 장치와 프로세스 쪽 장치를 나눠 봅니다.

아래 세 줄은 실습이에요. 같은 구조를 내 주제에 적용하는 실습 넷을 하고, ProcessGPT 시연을 본 다음, 학습 노트를 정리하면서 과정을 마칩니다.

이번 시간에는 그 가운데 앞의 세 단원, 현장 지식이 AI의 판단 근거가 되는 과정을 되짚습니다.

[SLIDE 04] 경보에서 추천안까지: 에이전트가 읽는 지식

목차의 첫 줄부터 시작하겠습니다. 첫 번째 단원은 경보에서 추천안까지, 에이전트가 읽는 지식이에요. 물음부터 하나 드릴게요. 에이전트가 받은 건 경보 한 건뿐이었습니다. 그런데 무엇을 읽어서 원인과 조치까지 담긴 판단을 만들었을까요? 지식 영역을 마무리할 때는 판단에 쓸 재료가 저마다 제자리에 놓여 있다는 데까지 정리했죠. 이번 회차에서는 거기서 한 걸음 더 갑니다. 클래스나 스키마의 정의를 처음부터 다시 외우자는 게 아니에요. 지식이 어떤 길로 에이전트의 입력이 되는지, 그 길을 따라가 보자는 겁니다.

[SLIDE 05] 따라갈 사건: 쿨러 성능 저하 경보에서 결과 보고까지

따라갈 건 앞 회차들에서 계속 다뤄 온 쿨러 사건이에요. HYD-01에서 오전 11시를 조금 넘겨 쿨러 성능 저하 경보가 났고요, 이 경보 한 건이 처리 건 하나를 열었습니다. 그 처리 건이 지나간 자리를 화면에 왼쪽부터 한 줄로 놓았어요.

맨 왼쪽이 경보입니다. 바로 옆 둘째 자리에서 에이전트가 원인을 진단하고 조치를 제안했죠. 셋째 자리에서는 운전원이 그 제안을 승인했어요. 승인 뒤로는 팬 속도를 올리는 냉각 명령이 나가고, 유온을 다시 읽어서 회복됐는지 확인합니다. 유온은 51도 남짓까지 내려왔고, 경보도 해제됐어요. 그다음 쿨러 핀을 세척하라는 작업지시가 등록되고, 결과 보고가 정상으로 나간 뒤 끝났습니다.

자리마다 아래에 작게 적힌 문자열 보이시죠? 그 자리에 남은 기록의 번호예요. 경보에도, 에이전트의 판단에도, 명령과 작업지시에도 번호가 붙어 있습니다. 번호를 읽을 필요는 없어요. 기억해 둘 건 자리마다 기록이 따로 남았다는 점입니다. 나중에 이 번호로 기록과 기록을 이어서 거슬러 볼 수 있거든요.

예를 들어 작업지시 기록에는 그 작업지시가 어느 판단에서 나왔는지, 판단 번호가 함께 적혀 있어요. 그래서 세척을 맡은 정비 담당자는 작업지시 한 장에서 출발해서, 왜 이 세척을 하게 됐는지 에이전트의 판단과 그 근거까지 따라갈 수 있습니다. 기록이 자리마다 남고 서로 이어져 있다는 게 이런 쓸모예요.

이 여덟 자리가 이번 회차 이론 전체의 지도예요. 지금까지 배운 지식, 에이전트, 프로세스가 어디에서 무슨 일을 했는지, 이 줄 위에서 하나씩 짚어 가겠습니다.

[SLIDE 06] 세 영역이 맡는 구간

같은 줄 위에 이번엔 세 영역을 겹쳐 놓았어요. 세 군데를 보시면 됩니다. 맨 위의 긴 괄호, 하늘색으로 칠한 둘째 자리, 그리고 왼쪽 아래의 상자예요.

맨 위 괄호가 프로세스입니다. 경보부터 끝까지 전체를 덮고 있죠. 어떤 순서로 무엇을 하고 각 자리를 누가 맡는지, 처음부터 끝까지 책임지는 쪽이 프로세스예요.

하늘색 둘째 자리가 에이전트입니다. 줄 전체를 봐도 에이전트가 일하는 곳은 이 한 자리뿐이에요. 원인을 진단하고 조치를 제안하는 데까지가 에이전트의 몫입니다. 자리로 세어 보면 더 분명해요. 여덟 자리 가운데 에이전트의 자리는 둘째 하나이고, 추천안이 나온 뒤에도 승인부터 끝까지 여섯 자리가 남아 있습니다. 추천안이 나온 순간은 업무가 끝난 때가 아니라, 이제 막 앞부분을 지난 때라는 거죠.

왼쪽 아래 상자는 판단에 쓸 지식과 값을 묶은 거예요. 지식 그래프, SOP와 판단 규칙, 그리고 시계열 DB와 업무 DB의 값이 여기 모여 있습니다. 그런데 이 상자가 줄과 어떻게 이어져 있는지 보세요. 에이전트 자리로 올라가는 화살표 하나뿐이에요.

이 그림은 에이전트가 판단 재료를 읽는 경로를 강조한 겁니다. 지식을 읽을 수 있는 쪽이 에이전트뿐이라는 뜻은 아니에요. 승인 조건을 확인하는 시스템도 필요한 기준을 읽습니다. 이 단원과 이어지는 두 단원에서는 바로 이 화살표 하나를 들여다볼 거예요. 지식이 어떤 모양으로 에이전트에게 건너가는지요. 에이전트 자리부터 끝까지, 프로세스가 맡은 구간은 뒤의 두 단원에서 다룹니다.

[SLIDE 07] 에이전트 단계의 입력: 경보 메시지 한 건

화살표를 따라가기 전에, 에이전트가 처음 손에 쥔 것부터 확인해 볼게요. 에이전트 단계가 열릴 때 주어지는 건 화면 왼쪽의 경보 메시지 한 건입니다. 여기에 뭐가 있고 뭐가 없는지 보면, 에이전트가 왜 도구를 불러야 하는지가 보여요.

들어 있는 것부터 볼까요? 맨 위가 이 경보 한 건을 가리키는 번호고요, 그 아래로 어느 설비인지, 어떤 조건이 성립했는지, 심각도와 경보 상태, 발생 시각이 있습니다. 맨 아래에는 판정할 때 쓴 값도 붙어 있어요. 경보가 난 순간의 유온과 냉각 효율이죠. 유온은 경보 조건에 걸릴 만큼 올라 있었고, 냉각 효율은 절반에 못 미치는 46퍼센트쯤이었습니다.

이번엔 없는 걸 볼게요. 원인이 뭔지는 없습니다. 쿨러 성능 저하라는 조건이 성립했다, 그것만 알려 줄 뿐이에요. 어떤 조치를 할 수 있는지도 없고, 무엇을 기준으로 조치를 견줄지도 없습니다. 지금 생산 오더의 납기가 얼마나 남았는지도 없고요.

값이 기준을 넘었다는 것과 왜 넘었는지 안다는 건 다른 일이에요. 경보는 앞의 것까지만 알려 줍니다. 에이전트가 쓰는 모델이 아무리 유능해도, 이 메시지만 보고 원인과 조치를 말하면 그건 짐작이에요. 모자란 건 전부 바깥에서 읽어 와야 합니다.

[SLIDE 08] 스킬이 정한 도구와 도구마다 읽는 재료

그럼 무엇을 어디서 읽을지는 누가 정할까요? 스킬이 정해 둡니다. 냉각 긴급 대응 에이전트는 냉각 긴급 대응 스킬을 따라 일하고, 그 절차에 부를 도구가 적혀 있어요. 화면 위쪽 띠가 그 도구들이고, 아래쪽 띠가 도구마다 읽는 재료입니다.

왼쪽 경보에서 출발해 볼게요. 스킬이 안내하는 첫 도구가 진단 도구입니다. 아래에서 파란 화살표 두 개가 올라오죠? 하나는 지식 그래프에서, 하나는 시계열 DB에서 와요. 원인 후보와 판별 기준은 그래프에서 읽고, 판단 시점의 센서 값은 시계열 DB에서 읽어서 원인을 가립니다.

다음 자리는 업무 DB를 읽는 도구 두 개예요. 생산 오더와 품질 로트를 읽어서 납기와 품질 사정을 확인합니다. 이 자리만 회색으로 그린 건, 스킬에 이 단계가 「필요하면 확인한다」고 적혀 있어서예요. 늘 부르는 도구가 아니라, 에이전트가 상황을 보고 부를지 정합니다.

그다음이 조치 평가 도구예요. 이 그림은 스킬의 안내 순서입니다. 실제 호출 순서나 업무 조회 여부까지 모든 실행에서 같다는 뜻은 아니에요. SOP와 판단 규칙, 그리고 성과 기준을 읽어서 조치 후보를 견줍니다. 마지막으로 제출 도구가 추천안을 제출해요.

아래 띠의 재료는 하나도 이 자리에서 만들어진 게 아니에요. 그래프의 관계는 스키마로 정의했고, 판별 기준과 SOP는 매뉴얼에서 뽑아 검토하고 적재했죠. 판단 규칙과 성과 기준도 앞에서 정해 둔 겁니다. 그리고 아래 띠 오른쪽에 업무 원천 읽기라고 적어 둔 것처럼, 여기 그린 업무 도구들은 설비와 업무 시스템의 재료를 읽어요. 쓰는 건 추천안 제출 하나뿐입니다. 그러니까 처음 물음의 답은 이거예요. 경보 한 건만 받은 에이전트는 스킬이 정한 도구로, 미리 정의하고 등록해 둔 지식과 지금의 값을 읽어서 판단을 만듭니다.

[SLIDE 09] 그래프 질의와 증거: 원인을 가리는 지식과 값

에이전트가 무엇을 어디서 읽는지는 이렇게 잡혔어요. 두 번째 단원은 그래프 질의와 증거입니다. 그 도구들 가운데 첫 번째인 진단 도구를 들여다볼 건데요, 이 도구가 처음 손에 쥔 건 경보의 패턴 이름 하나예요. 이 이름 하나에서 어떻게 원인 하나에 닿을까요? 그래프의 어디로 들어가서, 무엇을 읽고, 무엇과 견주는지, 진단 도구 한 번의 호출 안에서 따라가 보겠습니다.

[SLIDE 10] 경보의 패턴 코드에서 원인 후보까지

먼저 짚을 게 있어요. 경보 메시지와 지식 그래프는 서로 다른 곳에서 만들어졌습니다. 경보는 감지 서비스가 냈고, 그래프는 사람이 정의하고 적재했죠. 이 둘이 만나는 지점은 딱 값 하나예요.

경보의 패턴 칸에 쿨러 성능 저하를 뜻하는 코드가 적혀 있었죠. 그래프의 이상 패턴 노드에도 같은 코드가 속성으로 들어 있습니다. 진단 도구는 이 코드로 그래프의 출발 노드를 찾아요. 화면 맨 왼쪽, 진하게 칠한 동그라미가 그 노드입니다.

거기서부터는 관계를 따라가요. 이상 패턴이 증상을 감지하고, 증상이 고장 유형을 가리킵니다. 유온 상승이라는 증상을 거쳐서 쿨러 냉각 성능 상실이라는 고장 유형에 닿는 거죠.

그런데 마지막 화살표를 보세요. 방향이 반대예요. 원인이 고장 유형을 일으킨다는 관계라서, 원인 쪽에서 고장 유형 쪽으로 향합니다. 진단 도구는 이 화살표를 거슬러 올라가서 원인 후보를 얻어요. 후보는 셋입니다. 쿨러 핀 오염, 팬 구동 불량, 주변 온도 상승이죠. 앞에서 경보 메시지에는 원인이 없다고 했는데, 그 빈칸에 들어갈 수 있는 답들이 여기서 처음 나온 거예요.

만약 경보의 코드와 그래프의 코드가 달랐다면 어떻게 됐을까요? 진단 도구는 출발 노드부터 찾지 못했을 겁니다. 여러 저장소가 같은 대상을 같은 값으로 가리키도록 식별자를 맞춰 둔 까닭이, 바로 여기서 드러나는 거예요.

[SLIDE 11] 스키마가 정한 관계가 조회의 길이 된다

방금 화살표로 따라간 길이 실제로는 어떤 질의로 적힐까요? 화면 왼쪽이 진단 도구가 원인 후보를 찾을 때 그래프 DB에 보내는 Cypher 조회문입니다. 그 길을 네 줄로 적은 거예요.

첫 줄에 달러 표시가 붙은 자리가 있죠. 여기에 경보의 패턴 코드가 들어갑니다. 그 코드를 가진 이상 패턴 노드에서 출발하라는 뜻이에요. 둘째 줄은 감지 관계로 증상에, 셋째 줄은 가리킴 관계로 고장 유형에 간다는 뜻이고요. 마지막 줄은 일으킴 관계를 거꾸로 거슬러서 원인에 닿는다는 뜻입니다. 화살표 방향이 앞 줄과 반대로 적힌 게 보이시죠?

이 조회문은 에이전트가 그때그때 지어서 쓰는 게 아니에요. 진단 도구 안에 미리 정해져 있고, 에이전트가 하는 일은 경보의 패턴 코드를 넘겨주는 것뿐입니다. 같은 조회 조건과 같은 데이터라면 같은 후보 집합을 찾을 수 있죠. 그 뒤의 진단 결과는 센서 값이나 기준이 바뀌면 달라질 수 있습니다. 어느 길로 후보를 찾았는지 재현할 수 있다는 점이 이 조회의 가치예요.

여기 나온 이상 패턴, 증상, 고장 유형, 원인은 스키마에 정의한 클래스예요. 관계의 이름과 방향도 스키마에서 정했고요. 그러니까 길은 스키마가 정했고, 그 길을 따라 돌아오는 건 쿨러 핀 오염 같은 인스턴스입니다.

여기서는 관계를 설계한 것과 실제 데이터를 연결한 것을 나눠 보세요. 스키마에 관계를 정의했더라도, 적재된 데이터에 그 연결이 없으면 이 조회에서 후보가 빠질 수 있습니다. 반대로 관계를 잘못 설계하거나 질의를 틀리게 쓰면 검증 오류가 날 수도 있죠. 후보가 없다는 결과만으로 실제 원인이 없다고 결론 내리지 말고, 조회 조건과 적재된 연결을 함께 확인해야 합니다.

[SLIDE 12] 증거 기준과 현재 값: 진단 도구가 읽는 두 곳

후보 셋을 찾았으니 이제 어떤 후보를 근거가 지지하는지 봅니다. 화면 왼쪽은 판단 기준이고, 오른쪽은 그 기준에 대어 볼 관측값이에요. 같은 숫자처럼 보여도 역할이 다릅니다.

왼쪽은 수업에서 사용하는 설비 매뉴얼의 기준입니다. 쿨러 핀 오염을 지지하는 조건은 최근 냉각 효율 평균이 70퍼센트 미만이고, 팬 진동 평균이 초당 0.9밀리미터 미만인 경우예요. 냉각 효율은 냉각 성능이 떨어졌는지를 보고, 팬 진동은 다른 후보인 팬 구동 불량의 기준과 구별하는 데 씁니다. 팬 구동 불량 쪽은 같은 진동 기준 이상, 주변 온도 상승 쪽은 캐비닛 온도 35도 이상을 봅니다.

이 수치가 모든 설비에 통하는 진단 법칙은 아니에요. 이 수업 사례에 등록된 후보와 기준입니다. 진동이 낮다는 사실 하나로 팬의 모든 고장을 배제할 수도 없죠. 적용할 설비와 측정 조건을 확인한 기준이어야 판단에 쓸 수 있습니다.

오른쪽은 당시 판단에 사용한 관측값입니다. 냉각 효율은 46퍼센트쯤이고, 팬 진동은 기준 아래였어요. 캐비닛 온도도 30도가 안 됐습니다. 이 값을 각각의 조건에 대면, 등록된 후보 가운데 쿨러 핀 오염 쪽의 근거가 맞습니다. 다른 두 후보는 이때의 기준을 충족하지 못했고요. 여기서 얻는 것은 관측과 등록 지식에 근거한 진단입니다. 현장에 있을 수 있는 모든 원인을 검사했다는 뜻은 아닙니다.

HYD는 기준과 관계를 그래프에서 관리하고, 자주 바뀌는 관측값은 시계열 DB에서 읽도록 나눴습니다. 그래프에 시간과 값을 저장할 수 없어서가 아니에요. 기준을 누가 개정하는지와 측정값이 어디에서 갱신되는지를 분명하게 관리하려는 선택입니다. 기준은 검토를 거쳐 개정하고, 값은 읽은 시각과 측정 조건까지 확인해야 하죠. 값이 새로 왔다고 기준을 덮어쓰거나, 오래된 값을 현재 값처럼 쓰면 두 역할이 섞입니다.

원인 후보를 구별하는 이유도 분명해요. 핀 오염에는 세척이, 팬 문제에는 팬 점검이, 주변 온도 문제에는 환기 개선이 필요하니까요. 진단 도구는 관계로 후보를 찾고, 기준과 관측을 대조해 그때 지지되는 후보를 제시합니다. 어떤 판단이든 이 근거의 범위까지 함께 읽어야 합니다.

[SLIDE 13] 조치의 판단과 출처: 추천안이 되기까지

원인이 정해졌으니 다음은 조치예요. 세 번째 단원은 조치의 판단과 출처입니다. 쿨러 핀 오염이라는 원인 앞에서 무엇을 해야 할까요? 그리고 그 판단의 근거는 어디까지 거슬러 볼 수 있을까요? 조치 후보를 견주는 판단, 그리고 추천안에 근거가 남는 방식을 따라가 보겠습니다.

[SLIDE 14] 업무 DB에서 읽는 값: 납기와 품질

조치를 견주려면 설비 안의 값만으로는 모자라요. 스킬의 순서를 다시 떠올려 보면, 진단 뒤에 필요하면 업무 값을 조회하도록 안내했죠. 생산 오더의 납기, 그리고 품질 로트의 사정 같은 설비 바깥 사정이에요. 이 값들은 업무 DB에 있습니다. 화면에 네 줄로 정리해 놓았어요.

위의 두 줄은 에이전트가 부르는 업무 조회 도구예요. 첫 줄은 생산 실행 시스템, MES에 해당하는 업무 DB에서 HYD-01에 걸린 생산 오더를 읽습니다. 납기까지 남은 시간과 남은 수량 같은 거죠. 둘째 줄은 품질 관리 시스템, QMS에 해당하는 업무 DB에서 이 설비의 품질 로트를 읽어요. 고온 구간에 생산된 로트와 검사 비용 같은 겁니다.

셋째 줄, 강조해 둔 줄을 보세요. 이런 값은 계속 바뀌니까 지식 그래프에 값으로 넣어 두지 않았어요. 대신 업무 데이터베이스를 등록할 때, 어느 표의 어느 열에 이 값이 있고 어떻게 읽는지를 입력 데이터의 자리로 만들어 뒀습니다. 그러니까 지식에 등록된 건 값이 아니라, 값이 놓인 자리예요.

넷째 줄은 이 자리를 쓰는 또 하나의 도구, 곧이어 부를 조치 평가 도구입니다. 평가 도구는 에이전트가 조회해 둔 값을 넘겨받지 않아요. 이 자리를 따라 판단하는 순간의 값을 직접 읽어서 순위에

넣습니다. 쿨러 사건에서는 납기가 여섯 시간이 채 안 남은 오더, 늦어지면 시간마다 적지 않은 지연 보상이 붙는 계약, 그리고 출하를 기다리는 로트 수백 개가 읽혔어요.

그럼 업무 조회 도구는 왜 따로 있을까요? 같은 자리를 읽어서, 에이전트가 그날의 사정을 확인하고 추천안의 설명에 담기 위해서예요. 그리고 평가 도구가 읽은 값은 값마다 어느 시스템에서 어떻게, 언제 읽었는지가 처리 기록에 남습니다. 같은 설비에 같은 원인이라도 그날의 납기 사정에 따라 순위가 달라질 수 있고, 왜 달라졌는지도 기록에서 확인할 수 있는 거죠. 이런 값을 지식처럼 한 번 적어 두고 쓰지 않고, 판단할 때마다 새로 읽는 까닭입니다.

[SLIDE 15] 조치 평가 도구의 결과: 순위와 승인 역할

이제 조치 평가 도구를 부를 차례예요. 이 도구를 한 번 불러서 받은 결과가 이 표입니다. 앞 장 넷째 줄의 도구 이름에 카드라는 말이 들어 있었죠? 조치 후보 하나하나를 카드라고 부릅니다. 후보는 넷이고, 칸은 순위, 조치와 점수와 판단 규칙, 승인 역할 세 칸이에요.

후보는 어디서 왔을까요? 쿨러 냉각 성능 상실이라는 고장 유형에 이어진 SOP가 넷이었고, 그 넷이 그대로 후보가 됐습니다. 먼저 DMN 결정표로 적은 판단 규칙이, 지금 설비 상태에서 각 후보를 해도 되는지 가렸어요. 가운데 칸의 둘째 줄을 보세요. 원격으로 하는 위의 세 조치는 설비가 원격 자동으로 운전 중일 때만 할 수 있는데, 이 사례에서는 그 조건을 충족했습니다. 실제 허용 여부는 운전 모드만이 아니라 각 조치에 연결된 나머지 조건도 함께 확인한 결과예요. 맨 아래 핀 세척은 작업지시라서 운전 모드와 상관없이 남고요. 그다음 BSC 관점으로 정한 KPI, 곧 성과 기준이 남은 후보의 순위를 매겼어요. 앞 장에서 본 납기 사정도 여기서 점수에 들어갑니다.

맨 윗줄, 1순위가 팬 최대 운전이고 점수가 가장 높습니다. 납기처럼 생산을 유지하는 데 가치를 두는 성과 기준이 반영된 결과예요. 한 조건만으로 순위가 정해지는 것은 아닙니다. 부하를 낮추는 두 후보는 점수가 똑같아서 공동 2순위고요. 맨 아랫줄, 원인을 없애는 쿨러 핀 세척은 당시 평가 결과에서는 점수가 마이너스였습니다. 이 점수는 그때 정한 성과 기준으로 비교한 결과이지, 조치 자체가 언제나 가치 없다는 뜻은 아니에요. 그래서 4순위예요. 이 순위와 점수를 어떻게 계산했는지는 조치 판단을 다룬 회차에서 자세히 봤죠.

그런데 4순위라고 해서 버려지는 조치는 아니에요. SOP를 배울 때 즉시 완화 조치와 근본 조치를 나눴던 걸 떠올려 보세요. 팬 최대 운전은 지금 오르는 유온을 누르는 완화 조치고, 쿨러 핀 세척은 원인을 없애는 근본 조치입니다. 당장의 순위에서는 밀렸지만, 이 세척은 뒤에서 작업지시로 다시 나타나요.

이번에는 오른쪽 승인 역할 칸을 함께 보겠습니다. 위의 세 후보는 운전원이 승인하는데, 맨 아래 쿨러 핀 세척만 설비보전팀장이 승인해요. 설비를 세우고 정비하는 일이라 승인하는 사람이 다른 겁니다.

여기서 중요한 게 있어요. 누가 승인할지를 에이전트가 판단한 게 아니라는 겁니다. 승인 역할은 조치 방법에 붙어 있는 지식이고, 에이전트는 그걸 읽어서 결과에 실었을 뿐이에요. 실제로 승인할

수 있는지는 이 정책과 처리 건의 담당, 현재 조건을 함께 확인합니다. 추천안에 역할 이름이 들어 있다고 그것만으로 승인 권한이 생기지는 않아요.

[SLIDE 16] 도구 사이를 넘어가는 식별자

평가까지 마친 결과는 제출 도구로 넘어갑니다. 그런데 도구와 도구 사이로는 무엇이 넘어갔을까요? 지금까지 거친 도구 사이를 넘어 다닌 걸 화면에 한 줄로 놓았습니다. 칸마다 아래에 적힌 게 그 칸을 가리키는 식별자예요. 여기서 볼 건, 넘어가는 게 쿨러 핀 오염이라는 말이 아니라 이 식별자라는 점입니다.

맨 왼쪽이 경보예요. 경보의 식별자가 진단 도구에 들어가면 둘째와 셋째 칸, 고장 유형과 원인의 식별자가 나옵니다. 이 예의 식별자 접두사는 고장 유형과 원인을 구별하는 표기 관례예요. 접두사 자체가 데이터의 종류나 정확성을 보장하는 것은 아닙니다.

그 두 식별자가 조치 평가 도구에 들어가서 넷째 칸이 나옵니다. 방금 본 1순위 조치 방법, 팬 최대 운전의 식별자예요. 마지막으로 제출 도구가 이것들을 묶어서 추천안 하나로 제출하면, 그 추천안에 판단 번호가 붙습니다. 맨 오른쪽 칸이에요.

식별자 대신 말로 넘기면 어떻게 될까요? 쿨러 핀 오염과 이름이 비슷한 원인이 그래프에 하나 더 있다고 해 보죠. 뒤의 도구는 어느 쪽을 읽어야 할지 알 수가 없습니다. 식별자로 넘기면 앞의 도구가 가리킨 바로 그 노드를 뒤의 도구가 읽어요. 사람이 나중에 추천안을 거슬러 볼 때도 같은 식별자로 같은 대상을 찾아갈 수 있고요.

[SLIDE 17] 제출된 추천안에 담긴 것

그 판단 번호가 붙은 추천안을 열어 볼게요. 화면 왼쪽이 제출 도구로 낸 추천안을 항목별로 나눠 적은 거예요. 맨 위가 방금 본 판단 번호고요. 그 아래 항목들이 앞의 도구들이 읽고 계산한 결과를 어떻게 담고 있는지 볼게요.

원인은 쿨러 핀 오염입니다. 그 아래 근거에는 팬 진동은 정상인데 냉각 효율이 절반에 못 미치는 46퍼센트쯤으로 떨어져 있다는 값이 적혔어요. 진단 도구가 시계열 DB에서 읽은 값이죠. 배제 칸에는 맞지 않은 후보 둘, 팬 구동 불량과 주변 온도 상승이 적혔습니다. 왜 그 원인이 아닌지까지 남긴 거예요.

추천은 팬 최대 운전이고, 평가에서 받은 점수가 같이 실렸습니다. 그 아래 예측은, 이 조치를 하면 주어진 가정에서 유온이 50도 조금 아래로 내려갈 것으로 본 계산 결과고요. 예측값은 실제로 측정한 회복 온도와 구별해야 합니다. 예측이 좋아 보여도 실행 뒤에는 다시 관측해야 해요. 마지막 줄이 출처예요. 매뉴얼의 즉시 완화 조치 절을 가리킵니다.

승인하는 사람 입장에서 생각해 볼까요? 원인과 추천만 있다면 믿을지 말지 정할 근거가 없습니다. 근거가 된 값, 배제한 후보, 예측, 출처가 함께 있으니까, 무엇을 보고 이렇게 판단했는지 확인할 수 있는 거예요.

[SLIDE 18] 인제스천이 남긴 출처: 매뉴얼의 절에서 추천안까지

추천안 맨 아래의 출처는 어떻게 여기까지 실려 왔을까요? 판단하는 자리에서 새로 만든 게 아닙니다. 매뉴얼을 지식으로 옮기는 인제스천 과정에서 미리 남겨 둔 연결을 따라온 거예요. 화면의 다섯 칸을 왼쪽부터 따라가 볼게요.

첫 칸은 매뉴얼 원문입니다. 원인과 증거 기준은 원인과 증거를 다룬 절에, 팬 최대 운전 같은 조치는 즉시 완화 조치를 다룬 절에 적혀 있었어요.

둘째 칸에서 원문 문장을 읽어 지식 후보를 뽑고, 사람이 원문과 대조해서 검토하고 승인하는 절차를 거칩니다. 왜 굳이 사람이 볼까요? 앞에서 그래프의 기준은 여러 판단이 함께 쓴다고 했죠. 잘못 뽑힌 기준 하나가 적재되면, 그 뒤의 판단이 전부 그 기준을 따르게 됩니다. 그래서 들어가기 전에 한 번 거르는 거예요.

셋째 칸, 적재에서는 후보만 넣은 게 아니에요. 문서의 절을 노드로 두고, 작업 단계는 그 절을 참조한다는 관계로, 판단 규칙은 그 절에서 나왔다는 관계로 이어 뒀습니다. 앞에서 본 원인의 증거 기준도 마찬가지예요. 원인과 증거를 다룬 절과 원문 인용 구간이 붙은 채로 적재됐습니다. 출처는 조치에만 있는 게 아니라 원인을 가린 기준에도 있는 거죠.

넷째 칸의 HYD 판단 경로는 이 사례에서 문서를 새로 검색하는 대신 적재된 관계를 읽습니다. 규칙과 SOP를 관계를 따라 읽으니까, 그 규칙이 이어진 절이 같이 딸려 오는 거죠. 그래서 마지막 칸, 추천안의 출처에 즉시 완화 조치 절이 실린 겁니다.

출처가 있으면 승인하는 사람이 추천안에서 출발해서 매뉴얼의 그 절까지 거슬러 갈 수 있어요. 반대로 적재할 때 출처 관계를 안 남겼다면 어떨까요? 판단은 똑같이 나와도 근거 문서는 비어 있겠죠. 출처 연결은 판단을 검토할 길을 열어 줍니다. 링크가 있다는 것만으로 내용이 맞는 건 아니에요. 문서 판본과 적용 대상, 인용한 문장이 실제로 그 기준을 뒷받침하는지까지 대조할 수 있어야 합니다.

[SLIDE 19] 지식의 빈자리가 에이전트의 출력에 나타나는 모습

표는 지식이나 값이 비었을 때 먼저 살펴볼 곳입니다. 출력이 이상하다고 곧바로 에이전트의 문장부터 고치면, 원래 원인을 놓칠 수 있어요.

위 두 줄은 후보가 빠지는 경우입니다. 질의가 따라갈 데이터의 관계가 없거나, SOP와 고장 유형의 연결이 없으면 후보가 누락될 수 있어요. 조회가 정상 종료돼도 답이 빈 경우가 있다는 뜻입니다. 다만 빈 결과는 연결 누락만의 증거는 아니에요. 조회 조건, 접근 권한, 적재 대상도 살펴봐야 합니다. 반대로 형식 오류라면 검사에서 바로 거절될 수 있고요.

가운데 두 줄은 판단 근거를 충분히 확보하지 못한 경우예요. HYD 진단 도구는 증거가 부족하거나 데이터 신선도 검사를 통과하지 못하면 보류를 반환합니다. 스킬에도 모르는 값을 꾸미지 말고 보류 이유를 내도록 정해 뒀죠. 지침을 적었다고 모델이 언제나 지킨다고 믿는 대신, 도구의 검사와 실제 제출 결과를 함께 확인합니다.

보류가 나오면 무엇을 더 확인해야 하는지가 남아야 합니다. 원천 연결을 복구해야 하는지, 새 값을 기다려야 하는지, 기준을 다시 검토해야 하는지가 달라요. 재시도나 재평가를 할 때는 그 이유를 해결했는지 먼저 봅니다. 보류는 업무 완료와도 다르고, 틀린 값을 넣어 빈칸을 채우라는 요청도 아닙니다.

마지막 줄은 출처입니다. 규칙에서 원문 절로 이어지는 관계가 빠지면 추천은 나와도 그 기준의 근거를 추적하기 어렵습니다. 후보의 누락, 근거 부족, 출처 누락을 나눠 보면 확인할 자리도 구체적으로 좁힐 수 있어요.

[SLIDE 20] 지식을 고치는 일과 에이전트를 고치는 일

고칠 곳은 무엇이 바뀌었느냐에 따라 달라요. 화면 왼쪽이 지식을 고치는 경우, 오른쪽이 에이전트를 고치는 경우입니다.

매뉴얼이 개정돼서 기준값이 바뀌었다고 해 보죠. 조치 방법이나 승인 역할이 바뀐 경우도 마찬가지예요. 이때는 지식을 고칩니다. 개정된 문서를 다시 인제스천해서 검토하고 적재하면 돼요. 조회 방식과 데이터 계약이 유지된다면, 같은 진단 도구가 개정된 기준을 읽게 할 수 있습니다. 기준의 단위나 뜻, 관계 구조까지 바뀌었다면 도구와 명세도 함께 점검해야 해요. 적재 뒤에는 대표 사례를 다시 판단시켜 개정이 의도대로 반영됐는지 확인합니다.

반대로 일하는 순서나 결과의 모양을 바꾸고 싶을 때가 있죠. 업무 값을 먼저 읽게 한다든지, 추천안에 새 항목을 넣는다든지요. 이때는 스킬을 고치고, 명세와 시나리오로 바뀐 행동을 확인합니다. 그 변경이 읽는 지식의 구조에도 영향을 주는지는 함께 확인하고요.

여기서 흔히 하는 실수가 하나 있어요. 기준값이 바뀌었는데 스킬 문장에 새 기준을 적어 넣는 겁니다. 당장은 맞게 돌아갈 수도 있어요. 그런데 기준이 그래프와 스킬, 두 곳에 생기죠. 문서가 다시 바뀌면 두 곳이 어긋납니다. 그러니까 어느 곳이 기준의 정본인지 정하고, 에이전트와 도구가 그 기준을 읽게 해야 해요. 여러 곳에 복사본이 필요하더라도 판본과 갱신 책임을 관리해야 서로 다른 기준으로 판단하는 일을 줄일 수 있습니다.

[SLIDE 21] 읽어 온 것과 에이전트가 정한 것

앞의 세 단원을 닫기 전에, 추천안 한 건에 들어간 걸 한번 갈라 볼게요. 무엇이 읽어 온 것이고, 무엇을 에이전트가 정했을까요? 출처에 따라 카드 세 장으로 나눴습니다. 카드 아래 꼬리표는 그 내용이 언제 바뀌는지를 적은 거예요.

첫 카드는 지식에서 읽은 것입니다. 원인 후보와 증거 기준, 조치 후보와 판단 규칙, 승인 역할과 근거 문서의 절이에요. 승인된 기준이나 정책이 개정될 때 바뀝니다. 정적 지식이라는 말은 영원히 고정된다는 뜻이 아니에요.

둘째 카드는 원천에서 읽은 값이에요. 냉각 효율과 팬 진동, 생산 오더와 품질 로트죠. 이건 계속 바뀝니다.

셋째 카드가 에이전트가 스스로 정한 거예요. 어느 도구를 어떤 순서로 부를지, 근거를 어떤 문장으로 설명할지입니다. 업무 값 조회를 할지 말지, 한다면 언제 할지 같은 게 여기 들어가요. 그래서 실행할 때마다 달라질 수 있습니다.

정리하면, 실행마다 설명의 문장은 달라질 수 있어도, 후보와 기준, 그리고 읽어 온 값은 에이전트가 정하지 않는다는 거예요. 판단의 근거를 지식과 원천으로 확인할 수 있게 하고, 에이전트는 그걸 읽어서 엮는 역할입니다. 현장 지식이 AI의 판단 근거가 된다는 말은, 바로 이 구조를 가리키는 겁니다.

다음 시간에는 이 추천안이 운전원의 승인과 시스템 실행을 거쳐 끝난 업무가 되기까지를 따라갑니다.

[이론 02] 에이전트의 결과가 업무 완료로 이어지는 전체 구조 복습하기

[SLIDE 22] 제안에서 완료까지: 승인과 시스템 실행

이번 시간에는 에이전트의 결과가 업무 완료로 이어지는 전체 구조를 되짚습니다. 앞 시간에는 에이전트가 지식과 값을 읽어서 추천안 한 건을 제출하는 데까지 따라왔죠. 이번 시간의 첫 단원, 목차의 네 번째 단원은 제안에서 완료까지, 승인과 시스템 실행이에요. 그 추천안을 누가 승인하고, 어떤 시스템이 무엇을 실행해야 이 일이 끝났다고 말할 수 있을까요?

이어서 같은 구조를 내 업무에 적용할 실습과 제품 시연의 관찰 기준까지 함께 준비하겠습니다. 아직 실습을 수행한 상태는 아니에요. 무엇을 만들고 어떤 결과로 확인할지를 먼저 연결해 두고, 실제 실습 뒤에 이번 회차를 마무리합니다.

[SLIDE 23] 추천안이 제출된 시점의 처리 건

사건의 줄을 다시 가져왔습니다. 이번엔 추천안이 제출된 바로 그 순간이에요. 화면을 보면 왼쪽 두 자리만 진하고, 나머지는 회색이죠.

진한 두 자리가 경보, 그리고 에이전트의 진단과 제안입니다. 왼쪽 아래 지식 상자에서 올라온 파란 화살표가 에이전트 자리에 닿아 있죠. 지식과 값을 도구로 읽어서 추천안을 제출하기까지, 앞 시간에 따라온 일이 여기서 다 끝났어요. 위에 완료 구간이라고 적은 곳이죠.

그런데 이 순간 설비는 어떤 상태일까요? 팬은 여전히 60퍼센트로 돌고 있고, 유온은 계속 오르고 있습니다. 실제로 승인하는 순간엔 유온이 60도를 넘어 있었어요. 경보가 났을 때보다 몇 도나 더 오른 거죠. 추천안은 판단의 결과일 뿐이에요. 설비에는 아직 아무 변화도 일으키지 않았습니다.

회색으로 남은 미실행 구간이 승인, 냉각 명령, 재관측, 작업지시, 결과 보고예요. 누군가 이 자리들을 정해진 순서로 채워야 업무가 끝납니다.

[SLIDE 24] 승인에서 끝까지: 자리마다 맡는 쪽과 기준

이 자리는 누가 맡고, 무엇을 기준으로 채울까요? 추천안이 제출된 뒤에 남은 다섯 자리를 표로 놓았습니다. 가운데 칸이 그 자리를 맡는 쪽, 오른쪽 칸이 그 일의 기준이에요.

첫 줄, 승인은 운전원이 맡습니다. 기준은 조치에 정해진 승인 역할이에요. 팬 최대 운전의 승인 역할이 운전원이라서 운전원에게 간 거죠.

둘째 줄부터는 맡는 쪽이 모두 프로세스 엔진이에요. 냉각 명령은 엔진이 SCADA를 거쳐 PLC에 보냅니다. 기준은 승인된 조치, 곧 팬 속도 100퍼센트고요. 재관측은 엔진이 시계열 DB에서 값을 다시 읽는 일이에요. 유온이 55도 미만으로 내려오고 경보가 해제돼야 회복으로 봅니다. 조건을 둘로 둔 데는 까닭이 있어요. 값이 기준 아래로 한 번 내려간 것만 보고 회복이라고 하지 않으려는 거죠. 경보를 냈던 감지 쪽에서도 이상이 풀렸다고 해제를 내 줘야 하는 거예요.

작업지시 등록은, 회복을 확인한 뒤에 엔진이 정비 관리 시스템, CMMS에 등록해요. 마지막 결과 보고는 처리 건이 어느 끝에 닿았는지, 그리고 그때까지의 결과를 알리는 일입니다.

가운데 칸의 화살표 방향도 보세요. 명령과 작업지시는 엔진에서 바깥으로 나가는 쓰기고요, 재관측만 바깥에서 엔진으로 들어오는 읽기예요. 이 다섯 자리를 정해진 순서로 잇는 게 바로 프로세스입니다.

[SLIDE 25] 에이전트 단계의 끝: 제출하면 엔진이 이어받는다

그렇다면 에이전트 자리에서 첫 자리인 승인으로는 어떻게 넘어갈까요? 에이전트 쪽과 프로세스 쪽이 만나는 자리, 그게 제출이에요. 화면의 네 칸을 왼쪽부터 따라가 볼게요.

첫 칸에서 엔진이 에이전트 단계를 엽니다. 이 단계를 냉각 대응 에이전트에게 맡기죠. 둘째 칸에서 에이전트가 스킬과 도구로 추천안을 만들어요. 앞 시간에 본 진단, 업무 조회, 평가, 제출이 전부 이 칸 안에서 일어난 겁니다.

셋째 칸이 경계예요. 에이전트가 결과를 내면 그 단계는 제출됨 상태가 되고, 에이전트의 일은 거기서 끝납니다. 제출됨이란, 에이전트는 손을 뗐지만 엔진은 아직 이어받기 전인 짧은 틈이에요. 결과를 내는 일과 흐름을 앞으로 보내는 일이, 이 상태를 사이에 두고 둘로 나뉘어 있는 거죠. 넷째 칸에서 엔진이 제출된 결과를 확인하고, 결과를 프로세스 변수에 담은 다음, 운전원 승인 단계를 열어요.

눈여겨볼 점은, 에이전트가 다음 단계를 직접 열지 않는다는 거예요. 다음에 누가 무엇을 하는지 몰라도 돼요. 이 HYD 흐름에서 다음 단계를 여는 쪽은 엔진이고, 엔진은 프로세스 정의를 읽고 엽니다. 에이전트가 흐름을 마음대로 건너뛸 수 없는 이유죠.

[SLIDE 26] 출력 계약: 에이전트의 결과를 프로세스가 읽는 약속

엔진이 결과를 변수에 담는다고 했는데요, 그러면 무엇을 어떤 이름으로 담을지는 미리 약속돼 있어야겠죠? 처리 건이 들고 다니는 변수 가운데 넷을 골라 표로 놓았습니다. 왼쪽이 변수 이름, 가운데가 그 변수를 만든 단계, 오른쪽이 그 변수를 읽는 쪽이에요.

첫 줄의 추천안 변수는 에이전트의 진단 · 제안 단계가 만들었습니다. 이 변수가 운전원 승인 단계에 놓이고, 운전원은 그 안의 추천과 대안을 보고 결정해요.

둘째 줄의 승인 결과는 운전원 승인 단계가 만들고, 승인이냐고 묻는 게이트웨이가 읽습니다. 셋째 줄의 회복 여부는 재관측이 만들고, 유온이 정상이냐고 묻는 게이트웨이가 읽어요. 넷째 줄의 작업지시 번호는 작업지시 등록이 만들고, 결과 보고가 읽어서 보고에 싣습니다.

쿨러 사건에 대어 볼까요? 승인 결과에는 승인이 들어갔고, 그래서 승인 갈림은 냉각 명령 쪽으로 갔어요. 회복 여부에는 회복됐다는 값이 들어갔고, 그래서 다음 갈림은 작업지시 쪽으로 갔습니다. 만약 회복되지 않았다면 이 갈림은 작업지시 대신, 기준에 못 미쳤다는 결과 보고로 보냈을 거예요.

결국 앞 단계가 낸 이름을 뒤 단계가 그대로 찾아 읽는 구조예요. 게다가 엔진은 단계마다 내겠다고 선언해 둔 출력 항목만 처리 건 변수에 합칩니다. 에이전트의 결과 항목 이름이 선언과 다르면 필요한 값이 전달되지 않을 수 있어요. 그러면 뒤 단계에서 멈추거나 잘못된 기본값을 읽을 위험이 있죠. 이름뿐 아니라 형식과 의미도 명세대로 전달되는지 검사해야 합니다. 명세에서 정한 출력 계약이 에이전트 혼자 지키는 약속이 아니라, 프로세스와 맺은 약속인 이유예요.

[SLIDE 27] 승인 한 건이 처리 건에 남기는 값

그 약속대로 놓인 추천안을 운전원이 승인하면, 처리 건에 새 값이 남습니다. 표의 위 두 줄은 프로세스 변수고, 아래 두 줄은 처리 기록이에요.

첫 줄은 승인이라는 결정과 고른 조치입니다. 쿨러 사건에서는 추천안 그대로 팬 최대 운전을 골랐어요. 이 표는 그 선택을 같은 흐름의 변수와 기록 항목에 대응시킨 예입니다. 운전원 승인 단계가 채우는 값이죠.

강조한 둘째 줄이 승인한 역할이에요. 이 값은 승인하는 순간 서버가 채웁니다. 에이전트는 자기 결과에 이 값을 써넣을 수 없어요. 에이전트의 제안과 승인 처리 경로를 분리한 거죠. 수업 환경에서는 화면에서 고른 역할로 흐름을 경험합니다. 실제 업무에서는 로그인한 신원과 그 역할의 권한을 연결해서 확인해야 누가 승인했는지 책임을 물을 수 있어요.

아래 두 줄은 처리 기록입니다. 셋째 줄은 승인이 접수됐다는 기록이에요. 승인한 역할과 판단 번호가 같이 남습니다. 이 기록이 있어야 나중에 누가, 어떤 근거로, 무엇을 승인했는지 되짚을 수 있어요. 판단 번호로 추천안과 이어져 있으니까, 승인에서 추천안으로, 추천안에서 근거와 출처로 거슬러 갈 수 있는 거죠.

넷째 줄은 승인이 다음 단계로 전달됐다는 기록입니다. 이 전달이 성공한 뒤에야 냉각 명령이 시작돼요.

승인 시점의 조건도 다시 확인해야 합니다. 추천을 만든 뒤에 설비 상태나 정책이 바뀔 수 있으니까요. HYD도 현재 조건을 확인하지 못하거나 허용하지 않는 경우에는 그대로 실행에 넘기지 않도록 검사합니다. 과거의 추천이 있었다는 사실만으로 지금의 실행을 허용하지 않는 겁니다.

[SLIDE 28] 승인 뒤의 시스템 실행 셋

승인 뒤에 엔진이 연 서비스 작업 셋입니다. 왼쪽 운전원 승인에서 가운데 엔진으로, 엔진에서 오른쪽 세 곳으로 화살표가 이어지죠. 파란 화살표는 쓰기, 남색 화살표는 읽기예요.

오른쪽 맨 위가 냉각 명령입니다. 엔진이 SCADA를 거쳐 설비에 팬 속도 100퍼센트를 지시했고요, 설비는 실행을 마쳤다고 답했습니다. 인터록 검사도 통과했어요.

가운데가 재관측이에요. 이 화살표만 방향이 반대죠? 재관측은 쓰는 일이 아니라 값을 읽어 오는 일이라서 그렇습니다. 엔진이 시계열 DB에서 유온을 다시 읽었더니, 60도를 넘던 유온이 51도 남짓까지 내려와 있었어요. 회복 기준인 55도 아래로 들어왔고, 경보도 해제됐습니다. 이 판정은 명령 직후에 한 번 읽고 끝내는 게 아니에요. 팬을 올려도 유온이 내려오는 데는 시간이 걸리니까, 정해 둔 시간 동안 지켜본 다음에 판정합니다.

맨 아래가 작업지시 등록이에요. 팬을 올려서 온도는 내렸지만, 핀 오염이라는 원인은 그대로 남아 있죠. 그래서 엔진이 CMMS에 쿨러 핀 세척 작업지시를 등록했습니다. 이건 팬 최대 운전을 승인한 흐름에 설계된 후속 작업지시예요. 앞 표의 핀 세척 조치 자체를 직접 선택해 승인하는 경우와는 구별합니다. 작업지시를 등록했다는 사실도 현장에서 세척을 끝냈다는 뜻은 아닙니다. 그 정비의 실제 완료는 별도 결과로 확인해야 해요.

여기서 두 가지를 구분해 두세요. 명령이 실행됐다는 응답과 문제가 풀렸다는 확인은 다른 일이에요. 그래서 명령 뒤에 재관측이 따로 있습니다. 그리고 증상이 가라앉은 것과 원인이 없어진 것도 다르죠. 그래서 회복 뒤에도 작업지시가 남는 겁니다.

[SLIDE 29] 세 가지 결말: 승인, 거절, 응답 없음

여기까지는 운전원이 승인한 경우였어요. 승인하지 않으면 어떻게 될까요? 같은 추천안이라도 사람의 결정에 따라 결말이 셋으로 갈립니다. 표의 가운데 칸, 시스템 실행의 수를 견줘 보세요.

첫 줄, 이 사례의 정상 경로에서는 승인 뒤 명령이 실행되고, 회복을 확인한 다음 작업지시가 등록돼 정상 종료로 닫혔습니다. 승인만으로 정상 종료가 보장되는 건 아니에요. 회복 기준을 충족하지 못하면 미달 결과를 남기는 다른 경로로 갑니다. 결과 보고는 정상이고요.

둘째 줄, 거절하면 시스템 실행이 한 건도 없어요. 설비 명령도 작업지시도 나가지 않고, 거절의 끝으로 닫힙니다. 결과 보고에는 반려, 그리고 설비 명령은 내지 않았다는 사실이 남아요. 기록을 열어 보면 가지 않은 단계들, 냉각 명령과 재관측과 작업지시는 취소 상태로 남아 있고, 거절의 사유도 함께 남습니다. 어느 갈래로 갔는지가 기록에 그대로 보이는 거죠. 그런데 여기서 조심할 게 있

습니다. 이때도 처리 건의 상태는 완료로 표시돼요. 그러니까 일이 실제로 이뤄졌는지는 어느 끝에 닿았는지와 결과 보고로 읽어야 합니다.

셋째 줄, 응답이 없으면 정해 둔 기한이 지난 뒤 승인 지연 알림만 나갑니다. 처리 건은 자동으로 승인되지도, 취소되거나 거절되지도 않아요. 승인을 계속 기다립니다. 늦게 승인하더라도 그때의 조건을 다시 확인해 허용되는 경우에 실행을 이어갑니다.

왜 기한이 지나면 알아서 승인하거나 취소하게 두지 않았을까요? 자동으로 승인하면, 아무도 결정하지 않은 조치가 설비를 움직이게 됩니다. 사람의 승인을 둔 의미가 사라지죠. 반대로 자동으로 취소하면, 유온이 오르고 있는데 필요한 조치가 조용히 사라져요. 그래서 결정은 끝까지 사람에게 남겨 두고, 늦어졌다는 사실만 알려서 사람을 다시 부르는 겁니다.

세 경우 모두 에이전트의 추천안은 같았습니다. 결말을 가른 건 사람의 결정이에요. 그리고 그 결정에 따라 어디로 갈지는 프로세스 정의가 정해 둔 길입니다.

[SLIDE 30] 세 업무의 처리 건: 누구의 승인과 어떤 시스템 실행

지금까지는 쿨러 사건 하나로 봤어요. 다른 업무에서도 이 구분이 그대로 맞을까요? 프로세스 영역에서 다룬 세 업무를 한 표에 놓았습니다. 줄마다 시작 사건, 에이전트의 추천안과 승인하는 사람, 승인 뒤의 시스템 실행이 차례로 놓여 있어요.

첫 줄이 쿨러 사건이에요. 쿨러 성능 저하 경보로 시작해서, 냉각 대응 에이전트가 냉각 조치를 추천하고 운전원이 승인합니다. 승인 뒤에는 명령, 재관측, 작업지시가 이어지죠.

둘째 줄은 정기 점검입니다. 운전시간이 정비 주기에 가까워졌다는 사건으로 시작해요. 정비 계획 에이전트가 언제 정비할지를 추천하고, 승인은 설비보전팀장이 합니다. 승인 뒤에는 CMMS에 정비 오더가 등록되고, 생산팀에 공지 메일이 나가요.

셋째 줄은 재고 보충이에요. 예비품 재고가 재주문점 아래로 내려간 사건으로 시작합니다. 구매 에이전트가 발주 수량과 공급사를 추천하고, 구매 담당이 승인하면 ERP에 발주가 등록돼요. 이어서 공급사 통지와 입고 확인이 따라오고요. 참고로 수업 환경에서는 공급사와 실제로 거래하지 않습니다. 기업 시스템을 흉내 낸 시뮬레이션과 가상 메일 주소를 써요.

왼쪽 칸, 시작 사건도 한번 견줘 보세요. 첫 줄은 설비의 센서 값이 일을 시작시키고, 아래 두 줄은 운전시간과 재고 같은 업무 데이터가 시작시켜요. 수업에서는 버튼이 그 신호를 대신 내 주지만요. 그런데도 세 처리 건은 모두 같은 모양의 경보 한 건으로 열립니다. 무엇이 일을 시작시키든, 처리 건으로 들어오는 입구는 하나로 맞춰 둔 거죠.

시작 사건도, 에이전트도, 승인자도, 실행 시스템도 업무마다 다릅니다. 그런데 자리는 똑같아요. 에이전트는 둘째 자리에서 추천안을 내는 데까지만 합니다. 그 추천안을 정해진 사람의 승인으로 잇고, 승인된 뒤에 업무 시스템을 움직여서 일을 끝내는 건 프로세스예요. 업무가 달라져도 제안, 결정, 실행과 결과 확인을 나눠 설계하는 원리는 사용할 수 있습니다. 모든 업무가 반드시 같은 단계 수나 같은 승인 방식을 가져야 하는 것은 아니에요.

[SLIDE 31] 에이전트 쪽 장치와 프로세스 쪽 장치

마지막 이론 단원으로 넘어가죠. 주제는 에이전트 쪽 장치와 프로세스 쪽 장치예요. 에이전트 영역에서는 스킬, MCP, 명세, 시나리오를 배웠고, 프로세스 영역에서는 정의, 인스턴스, 승인, 시스템 실행을 배웠죠. 같은 처리 건 안에서 이 장치들은 각각 무엇을 맡고, 무엇을 지킬까요? 지금까지 사건의 자리를 따라왔다면, 이번엔 같은 처리 건을 그 안에서 일한 장치의 눈으로 보겠습니다.

[SLIDE 32] 처리 건 한 건에 겹쳐 있는 두 쪽의 장치

배운 장치를 두 쪽으로 나눴습니다. 화면 왼쪽이 에이전트 쪽, 오른쪽이 프로세스 쪽이에요. 아래 꼬리표는 각 쪽이 지키는 걸 한마디로 적은 겁니다.

왼쪽, 에이전트 쪽에는 에이전트 실행, 스킬, MCP가 있고, 명세를 기준으로 개발하는 SDD와, 구체적인 행동 예로 기대 결과를 합의하고 검증하는 BDD가 있어요. 이 장치들은 한 단계 안에서 추천안 하나를 제대로 만들게 합니다. 그래서 지키는 건 판단의 품질이에요.

오른쪽, 프로세스 쪽에는 BPMN으로 그린 프로세스 정의와 그 정의로 열린 인스턴스, 사람의 승인인 HITL, 그리고 시스템 실행이 있습니다. 이 장치들은 단계와 단계를 이어서 업무를 끝내요. 지키는 건 순서와 책임입니다.

두 꼬리표를 쿨러 사건에 대어 보면 이렇습니다. 판단의 품질은 등록된 근거가 지지하는 원인을 가리고, 근거와 배제한 후보, 출처까지 추천안에 남긴 것이에요. 순서와 책임은 운전원이 승인하기 전에는 팬이 움직이지 않았고, 누가 승인했는지가 기록으로 남은 것이고요.

그리고 두 쪽은 서로를 대신하지 못합니다. 스킬을 아무리 잘 써도, 승인 없이 명령이 나가는 걸 막을 수는 없습니다. 프로세스를 아무리 잘 그려도, 에이전트가 근거 없는 원인을 내는 걸 막을 수는 없고요. 결과가 기대와 다를 때 어느 쪽 몫인지부터 가려야 하는 이유예요.

[SLIDE 33] 두 쪽의 장치가 이 처리 건에서 한 일

실제 처리 건에서 이 장치들은 어디서 일했을까요? 같은 사건의 줄 위에 두 쪽 장치가 일한 자리를 표시했습니다. 위쪽이 프로세스 쪽, 아래쪽이 에이전트 쪽이에요.

아래쪽부터 볼게요. 파란 상자로 두른 둘째 자리가 에이전트 쪽입니다. 냉각 대응 에이전트가 이 한 자리에서 냉각 긴급 대응 스킬을 따라 일했고요, 판단 엔진과 업무 DB의 MCP 도구를 사용할 수 있도록 연결했습니다. 지식 그래프는 판단 엔진의 진단 도구가 안에서 읽습니다. 이 처리 건의 요약에는 도구 호출 열 번이 남아 있어요. 다만 이 정본 사건의 호출별 상세 기록은 없어서, 연결된 도구를 모두 실제로 호출했다고 단정하지는 않습니다.

왼쪽 아래 문서 그림은 명세와 시나리오예요. 이건 실행 중에 일한 게 아닙니다. 결과에 무엇이 있어야 하는지, 어떤 상황에서 어떻게 행동해야 하는지를 실행에 앞서 정해 둔 거죠.

이번엔 위쪽이에요. 남색 긴 괄호가 프로세스 정의와 인스턴스입니다. 냉각 긴급 대응 정의에는 작업 아홉, 분기 둘, 타이머 하나가 그려져 있고요, 그 정의로 열린 인스턴스 하나가 경보에서 정상

종료까지를 덮었습니다. 정의는 하나지만, 경보가 날 때마다 인스턴스가 새로 하나씩 열려요. 쿨러 성능 저하 경보가 하루에 세 번 나면 처리 건도 셋이 되고, 저마다 따로 진행되고 따로 기록이 남습니다. HYD는 같은 테넌트와 프로세스 정의, 사건 식별자의 중복 시작을 검사합니다. 이것만으로 외부 명령이 반드시 한 번만 실행된다고 보장하는 것은 아니에요. 명령 재시도와 중복 효과는 별도 확인이 필요합니다.

괄호 아래를 보면, 승인 위의 선이 운전원 역할로 남은 승인 기록이고, 냉각 명령에서 작업지시까지 걸친 파란 괄호가 설비 명령과 작업지시를 남긴 시스템 실행이에요.

한눈에 보이시죠? 에이전트 쪽 장치는 줄의 한 자리에만 있고, 프로세스 쪽 장치는 줄 전체를 덮습니다.

[SLIDE 34] 스킬이 정한 순서와 프로세스 정의가 정한 순서

이 차이는 순서를 적은 문서에서도 드러납니다. 순서를 적은 문서는 둘이에요. 화면 왼쪽의 스킬, 오른쪽의 프로세스 정의입니다. 둘 다 순서를 정하지만, 다루는 범위와 지키는 방식이 달라요.

스킬은 에이전트가 읽는 지침입니다. 한 단계 안에서 어떤 도구를 어떤 순서로 부를지를 적어요. 냉각 긴급 대응 스킬에는 진단, 업무 값 조회, 대안 평가, 제출이 적혀 있죠. 업무 값 조회처럼 필요할 때만 하는 단계도 있어서, 에이전트가 상황에 맞게 고릅니다.

프로세스 정의는 엔진이 읽는 설계도예요. 단계와 단계 사이의 순서를 정합니다. 제안 뒤에 승인, 승인 뒤에 명령, 그다음 재관측과 작업지시죠. HYD 엔진은 지원하는 정의와 조건을 해석해 실행하고, 승인이라는 조건을 지나지 않으면 다음으로 넘어가지 않아요.

그럼 스킬에 승인을 받은 뒤에 명령하라고 적어 두면 어떨까요? 에이전트가 그 문장을 따를 수는 있습니다. 하지만 반드시 지킨다는 보장은 없어요. 지침은 읽는 쪽이 따르는 거니까요. 반면에 단계 사이의 순서는 엔진이 지킵니다. 이 예에서는 승인 경로 검사와 실행 권한 검사까지 갖춰 승인 없이 명령으로 넘어가는 길을 막습니다. 그림에 사람 작업을 하나 넣은 것만으로 통제가 완성되지는 않아요.

그렇다고 스킬의 순서가 쓸모없다는 말은 아니에요. 업무 값을 읽을지 말지처럼 그때 상황을 보고 정해야 하는 순서는 오히려 스킬에 두는 게 맞습니다. 프로세스도 분기 조건을 둘 수 있지만, 모든 세부 조회를 고정해 관리할 필요는 없으니까요. 반드시 지킬 순서는 정의에, 상황에 따라 고를 순서는 스킬에, 이렇게 나눠 두는 겁니다.

[SLIDE 35] MCP의 읽기와 프로세스의 쓰기

외부 시스템에 닿는 길도 두 쪽이 다르게 나뉘어 있습니다. 왼쪽은 에이전트가 MCP로 연결된 도구를 쓰는 길, 오른쪽은 프로세스가 서비스 작업으로 시스템을 부르는 길이에요.

왼쪽부터 보죠. 이 HYD 에이전트에 제공한 업무 도구는 주로 지식과 값을 읽는 도구입니다. MCP 자체가 읽기 전용 규약인 것은 아니에요. 쓰기 도구도 연결할 수 있어서 어떤 기능과 권한을 제공할지가 설계의 핵심입니다. 진단, 조치 평가, 생산 오더와 품질 로트 조회가 그렇죠. 쓰기는 추천안

제출 하나뿐이에요. 그 제출도 판단 기록 한 건을 남기는 일일 뿐, 설비나 업무 시스템을 바꾸지는 않습니다. 그리고 제출 결과에 허용되지 않은 실행 지시가 들어오는지도 정해진 계약으로 검사합니다. 모든 위험한 문장을 알아보는 만능 검사라고 이해하면 안 됩니다.

오른쪽, 설비와 업무 시스템을 실제로 바꾸는 쓰기는 프로세스의 서비스 작업에 있어요. 팬 속도를 100퍼센트로 바꾸는 냉각 명령, 쿨러 핀 세척 작업지시 등록이 그렇습니다. 이 작업은 승인이 전달된 뒤에만 열려요.

이렇게 나눠 두면 에이전트가 판단을 잘못해도 그 판단만으로는 아무것도 바뀌지 않습니다. 승인 단계에서는 추천의 근거와 현재 조건을 검토해야 하고요, 승인되지 않은 추천안으로는 쓰기에 닿을 길이 없어요. 그러니까 읽기와 쓰기를 가르는 경계가 곧 책임의 경계입니다.

[SLIDE 36] 명세와 시나리오가 확인하는 범위: 처리 건 전체의 행동

이 경계가 실제로 지켜지는지 확인하는 장치를 에이전트 영역에서 이미 써 봤죠. 거절 상황을 적은 시나리오예요. 화면 왼쪽처럼 Given, When, Then으로 나눠 적었습니다. 이 시나리오가 무엇을 확인하는지 다시 보겠습니다.

Given은 전제예요. 에이전트가 추천안을 내서 승인 단계가 열려 있는 상태죠. 이 전제가 서려면 에이전트 단계가 끝나고, 명세의 출력 계약대로 추천안이 놓여 있어야 합니다.

When은 확인하려는 행동이에요. 승인 권한이 있는 담당자가 사유를 적고 거절합니다. Then은 기대 결과고요. 설비 명령이 만들어지지 않고, 업무 처리도 일어나지 않고, 처리 건이 거절로 닫혀야 해요.

Then부터 아래 세 줄을 다시 보세요. 에이전트의 문장이 어땠는지는 하나도 묻지 않습니다. 전부 처리 건 전체가 어떻게 움직였는지를 묻죠. 이 예의 명세에는 출력 계약뿐 아니라 승인 전 실행 금지 같은 요구도 들어갑니다. 시나리오는 그 요구가 실제 행동으로 지켜지는지 확인할 예를 제공하죠. SDD나 BDD가 에이전트에만 쓰이는 방법은 아닙니다. 무엇을 유지하고 무엇을 바꿀지 명세로 합의하고, 정상·거절·실패 상황을 실행해 결과와 부수 효과를 확인하는 데 가치가 있어요. 시나리오 문서를 써 놓은 것과 그 검증을 통과한 것은 구별해야 합니다.

[SLIDE 37] HITL을 지키는 검사: 승인 없는 실행 경로

시나리오가 확인한 거절의 길은 사람의 결정이 흐름 안에 있어야 성립합니다. 사람의 승인, HITL은 프로세스 그림에서 사람 작업 하나로 그려지는데요, 그 자리가 그림에 반드시 있도록 지키는 검사가 셋 있어요. 화면의 카드 세 장입니다.

앞의 두 카드는 정의를 등록하기 전에 하는 검사예요. 첫째, 설비 명령 앞의 경로에 사람의 승인이 없으면 그 정의는 등록되지 않습니다. 둘째, 승인 조건을 지나지 않고 실행에 닿는 길이 하나라도 있으면 역시 등록되지 않아요. 거절 갈래에서 명령으로 가는 선을 잘못 그린 경우가 여기 걸립니다.

셋째 카드는 실행 중에 하는 검사예요. 승인하려는 사람이 그 조치의 지정 역할이나 허용된 역할 조건과 작업 담당 조건을 충족하지 못하면, 승인은 거부되고 거부됐다는 사실이 기록으로 남습니다. 예를 들어 운전원이 설비보전팀장 승인이 필요한 조치를 고르면, 그 역할을 임의로 대신할 수는 없습니다.

사람의 승인을 두자고 말로만 약속하면, 누군가 그림을 고치다가 빠뜨릴 수 있어요. 그 실수가 실행 중에야 드러나면, 그땐 이미 명령이 나간 뒤일 수도 있죠. 그래서 그림을 등록하는 순간과 실행하는 순간에 기계적으로 검사하는 겁니다.

[SLIDE 38] 같은 위험을 두 자리에서 막는 장치의 짝

이렇게 위험을 막는 장치는 에이전트 쪽에도 있습니다. 두 쪽의 장치를 위험별로 짝지어 봤어요. 윗줄이 에이전트 쪽 장치, 아랫줄이 프로세스 쪽 장치입니다. 그리고 둘을 잇는 세로선에 붙은 이름이, 두 장치가 함께 막는 위험이에요.

왼쪽 짝이 막는 건 근거가 모자란 추천안입니다. 도구는 증거와 값의 상태를 검사하고, 에이전트에는 근거가 부족하면 이유를 적어 보류하도록 지침을 둡니다. 사람도 추천안의 근거를 검토하도록 해서 놓친 부분을 다시 살펴요.

가운데 짝은 승인 없는 쓰기예요. 에이전트의 업무 도구 접근을 읽기와 제안 제출로 제한하고, 프로세스에서는 승인 조건의 갈림을 지나야만 실행에 닿게 합니다.

오른쪽 짝은 드러나지 않은 채 길어지는 대기예요. 에이전트 쪽에서는 실행 시간과 재시도에 한도가 있어서, 멈춘 작업이 드러나고 사람이 닫을 수 있습니다. 프로세스 쪽에서는 승인 지연 타이머가 사람의 응답이 늦어졌다는 걸 알려요. 타이머가 알림을 보냈다고 대기가 자동 해결되지는 않습니다. 누가 확인하고 어떻게 다시 처리할지도 정해야 하죠.

이 짝이 말하는 건 한 장치만 믿지 않는다는 거예요. 에이전트 쪽 장치는 모델의 판단에 기대는 부분이 있어서 놓칠 수 있습니다. 프로세스 쪽 장치는 그 판단의 결과가 효과를 내기 전에 한 번 더 거르고요.

[SLIDE 39] 결과가 기대와 다를 때 먼저 살펴볼 영역

역할을 이렇게 나눠 두면, 문제가 생겼을 때 어디를 볼지도 좁혀집니다. 결과가 기대와 다를 때 드러나는 증상 여섯을 표로 놓았어요. 가운데 칸이 먼저 살펴볼 곳, 오른쪽 칸이 그 영역입니다.

위의 네 줄, 지식과 에이전트의 몫은 앞 시간에 본 구분 그대로예요. 기준값이나 승인자가 틀리면 지식을, 읽을 값을 읽지 않았거나 결과 항목의 이름이 다르면 스킬과 명세를 봅니다.

이번에 더한 건 아래 두 줄, 프로세스의 몫이에요. 승인 없이 다음 단계가 열렸다면 프로세스 정의의 갈림 조건을 살펴요. 끝났는데 결과를 알 수 없다면 끝 이벤트와 결과 보고를 봅니다. 거절된 처리 건도 상태는 완료로 표시됐으니까, 상태만 봐서는 일이 이뤄졌는지 알 수 없었죠.

증상에서 영역부터 가리면, 고치지 않아도 될 곳을 건드리지 않게 됩니다. 예를 들어 승인자가 틀리면 조치의 정책부터 보되, 정책이 맞다면 역할 연결이나 작업 담당도 확인합니다. 표는 첫 조사 지점이지 원인을 한 곳으로 확정하는 답안은 아니에요. 실제로 값이 어디에서 잘못 넘어갔는지 기록을 따라 확인합니다.

[SLIDE 40] 세 영역이 주고받는 것

마지막으로 세 영역이 주고받는 걸 한 그림으로 묶었습니다. 지식, 에이전트, 프로세스, 세 상자를 왼쪽부터 놓았어요. 상자 사이의 파란 화살표가 영역 사이를 넘어가는 것입니다.

지식에서 에이전트로는 재료가 넘어가요. 관계와 기준, 그리고 값이 놓인 자리죠. 에이전트에서 프로세스로는 추천안 한 건이 넘어갑니다. 추천과 대안, 그리고 그 근거가 담겨 있고요.

프로세스는 그 추천안을 사람의 승인과 시스템의 실행으로 잇습니다. 그리고 누가 무엇을 결정하고 실행했는지를 아래의 띠, 처리 기록과 감사 기록으로 남겨요. 처음에 작업지시 한 장에서 판단과 근거까지 거슬러 갈 수 있다고 했죠. 그게 가능한 건 이 띠가 끝까지 이어져 있기 때문입니다.

그림은 판단 재료와 추천안이라는 두 연결을 강조한 겁니다. 실제 시스템에는 식별자와 상태, 오류나 보류 정보도 함께 오갑니다. 항목 이름과 형식만 같다고 서로 바꿔 끼울 수 있는 것은 아니에요. 값의 의미와 권한, 실패할 때의 행동까지 계약으로 맞추고 검증하면, 한 영역을 바꿀 때 다른 영역에 미치는 영향을 줄일 수 있습니다.

[SLIDE 41~42] 실습: 내 주제의 업무 명세 작성

지금까지 쿨러 사건 한 건 위에서, 지식이 에이전트의 판단이 되고 그 판단이 사람의 승인과 시스템 실행을 거쳐 끝나는 길을 처음부터 끝까지 다시 따라왔죠. 이제 그 구조를 직접 세워 볼 차례입니다. 이번엔 쿨러가 아니라 각자의 업무 위에서요. 내 업무로 옮기는 실습은 넷이 이어지는데, 첫 실습에서는 내 주제의 자료와 업무 목표, 완료 조건을 정의합니다. 먼저 각자에게 친숙한 작은 업무 하나를 고르게 돼요. 회의 준비나 자료 정리처럼, 매번 비슷하게 되풀이되는 일이면 좋습니다.

고른 업무를 화면의 카드 네 장에 맞춰 적어요. 첫 카드는 무엇을 받아 시작하고 어디서 정보를 읽을 수 있는지, 둘째는 에이전트가 무엇을 내는지, 셋째는 어느 결과를 누가 확인하는지, 마지막 카드는 무엇이 되면 끝났다고 보는지예요. 에이전트가 할 일은 읽기, 분류, 추천 가운데서 이 회차 안에 끝까지 만들 수 있는 크기로 줄입니다.

이 실습에서 남는 건 업무 명세 한 장이에요. 네 칸이 모두 차고, 마지막 카드의 완료 조건이 확인할 수 있는 문장이면 성공입니다. 「잘 정리된다」가 아니라, 무엇이 남으면 끝인지가 보여야 해요. 하지 못한 일을 어떻게 남길지도 적혀 있어야 하고요. 이 명세 한 장이 뒤의 세 실습이 따를 기준이 되거든요. 뒤의 실습에서는 이 명세부터 적습니다. 이어지는 지식 구조와 에이전트, 흐름이 무엇을 위해 필요한지 판단할 기준이 되는 한 장이에요.

[SLIDE 43~44] 실습: 내 문서로 만드는 작은 지식 구조

두 번째로 준비할 실습은 내 문서에서 주요 정보를 뽑아 작은 지식 구조를 만드는 일입니다. 첫 실습에서 작성할 명세에는 정보를 어디서 읽을지 적게 됩니다. 쿨러 사건에서 에이전트가 지식 그래프를 읽었던 것처럼, 내 에이전트에게도 읽을 지식이 있어야 합니다.

화면의 카드 네 장을 보세요. 입력은 명세에 적은 정보원 가운데 문서 하나예요. 제공된 문서도 좋고, 공개해도 되는 내 자료도 좋아요. 그 문서로 답하고 싶은 질문도 함께 정해 둡니다. 범위는 그 질문에 필요한 핵심 개념과 속성, 관계까지예요. 문서 전체를 옮기는 게 아니라, 질문에 답하는 데 필요한 만큼만 가려 냅니다.

남는 건 셋입니다. 개념과 관계를 정의한 스키마, 문서에 나오는 대상을 적은 샘플 인스턴스, 그리고 질문의 조회문이에요. 인스턴스마다 식별자를 붙이고, 어느 문서의 어느 대목에서 왔는지 출처도 붙입니다. 성공 기준은 마지막 카드예요. 만든 지식을 수업의 기준 그래프와 섞이지 않게 따로 둔 이름 공간에 넣고, 정한 질문이 조회로 풀리면 성공입니다. 여기서 만든 지식이 그다음 실습에서 만들 에이전트의 입력이 돼요. 문서의 답과 조회 결과가 같은지 비교해야, 데이터가 들어갔다는 확인에서 한 걸음 더 나아가 질문에 답할 수 있는 지식인지 확인할 수 있습니다.

[SLIDE 45~46] 실습: 내 지식을 읽어 제안하는 에이전트

세 번째 실습에서는 만들어 둘 지식 구조를 읽는 에이전트를 붙입니다. 앞 실습에서 질문이 조회로 풀리는지 확인한 뒤에, 그 조회 결과를 사용해 제안하게 할 거예요.

화면의 카드 네 장을 보세요. 입력은 앞에서 만든 스키마와 자료, 그리고 쓸 수 있는 읽기 도구예요. 범위는 에이전트에 목표와 스킬을 적고, 읽기 도구를 MCP로 연결하는 데까지입니다. 에이전트는 바이브 코딩으로 구성해요.

남는 건 에이전트 정의와 스킬, 그리고 제안이나 보류가 담긴 결과 한 건이에요. 이 에이전트가 지킬 규칙은 하나입니다. 확인된 정보로만 제안하고, 확인하지 못한 값이나 권한 밖의 작업은 지어내지 않고 보류로 냅니다. 예를 들어 회의 시간을 추천하는 에이전트라고 해 보죠. 일정을 조회할 도구가 연결되지 않았다면 빈 시간을 짐작해서 적으면 안 돼요. 후보를 낼 수 없다고 보류해야 합니다. 그래서 성공 기준은 마지막 카드의 두 가지예요. 제안의 근거가 실제로 읽은 값이고, 확인하지 못한 것은 보류로 남았으면 성공입니다. 근거가 있는 경우와 값을 읽지 못한 경우를 나눠 실행해 보면서, 스킬에 적은 규칙이 실제 결과에서도 지켜지는지 확인합니다.

[SLIDE 47~48] 실습: 직접 그린 흐름으로 내 에이전트 실행

네 번째 실습은 직접 그린 흐름으로 에이전트를 실행하는 일입니다. 앞 실습에서 만들 에이전트는 제안이나 보류 한 건을 내는 데서 멈춥니다. 쿨러 사건에서도 에이전트의 일은 추천안을 제출하면서 끝났고, 그 뒤를 이은 건 프로세스였죠. 내 업무도 마찬가지예요. 에이전트가 낸 제안이나 보류를 누가 확인하고 어떻게 끝낼지는 흐름이 정합니다.

작은 업무의 흐름은 bpmn.io로 그리고요, 화면의 다섯 칸, 시작, 에이전트 작업, 필요한 자리의 사람 확인, 결과 보고, 종료입니다. 그린 흐름을 준비된 연결 방식의 범위에서 등록하고 실행하면, 앞에서 만든 에이전트가 그 단계에서 일하고 처리 건 하나가 기록으로 남아요. 등록된 흐름과 그 처리 건의 기록이 이 실습에서 남는 것입니다.

이때 지킬 게 하나 있습니다. 일정 등록이나 메일 발송처럼 준비된 범위 밖의 외부 시스템 쓰기는 성공한 것처럼 적지 않아요. 하지 못한 일로 결과 보고에 남깁니다. 처리 건이 끝 이벤트에 닿아 닫히고, 사람 확인이 필요한 자리에 실제로 있었고, 결과 보고에 한 일과 하지 못한 일이 함께 남았으면 성공이에요. 단, 등록할 수 있는 연결이 먼저 준비돼 있어야 합니다. 수업 예제에는 강사가 실제 도구와 결과 형식에 맞게 채울 자리도 있어요. 연결이 없는 기능을 성공으로 꾸미지 말고, 확인 가능한 범위와 막힌 이유를 결과로 남겨야 합니다.

[SLIDE 49] 실습: ProcessGPT 시연

이제 ProcessGPT 시연을 볼 기준을 준비하겠습니다. 앞에서 안내한 네 실습은 작은 업무 하나를 명세, 지식, 에이전트, 흐름으로 연결하는 과정이었죠. 그럼 같은 구조가 실제 제품에서는 어떻게 나타날까요? 시연으로 지식, 에이전트, 프로세스의 연결을 확인하기 전에, 몇 장으로 짚고 시작할게요.

[SLIDE 50] BPMN 기반 멀티 에이전트 오케스트레이션: ProcessGPT

ProcessGPT는 BPMN으로 업무 흐름을 표현하고 여러 에이전트의 일을 조율하는 오픈소스 프로젝트입니다. 개발사의 공식 소개에서 확인할 수 있는 방향이에요.

여기서 볼 핵심은 제품 이름보다 오케스트레이션이라는 역할입니다. 여러 수행자가 각자의 일을 하더라도, 누가 언제 시작하고 어떤 결과를 다음에 넘길지는 조율해야 하죠. HYD에서 프로세스 엔진이 맡았던 역할입니다. 수행자가 여러 전문 에이전트로 늘어나도, 이 연결과 책임을 설계해야 한다는 점은 같습니다.

공식 소개에는 자연어로 프로세스를 정의하는 기능도 나옵니다. 업무를 말로 설명해 초안을 만드는 길이 열리는 거예요. 그렇다고 생성된 그림이 내 업무의 완료 조건이나 승인 규칙까지 자동으로 맞췄다는 뜻은 아닙니다. 사람이 의도를 검토하고 실제 실행 결과를 확인해야 하죠.

시연에서는 기능의 수보다 그 연결을 보겠습니다. 일을 맡기는 설계는 어디에 있고, 실제로 진행되는 한 건은 어디에 있으며, 결과를 낸 뒤 다음 차례를 누가 여는지요. 이 질문은 다른 제품을 볼 때도 그대로 사용할 수 있습니다.

[SLIDE 51] 이 과정에서 만든 것이 제품에서 나타나는 자리

우리가 만든 건 제품의 어느 자리에 해당할까요? 짝을 맞춰 보겠습니다. 표의 왼쪽 칸이 이 과정에서 만든 것, 가운데 칸이 ProcessGPT에서 그게 나타나는 자리, 오른쪽 칸이 그 구실이에요.

첫 줄은 판단에 필요한 지식의 자리입니다. 제품의 공개 구조에는 지식 그래프뿐 아니라 문서 검색을 위한 구성도 있어요. 이 표는 HYD 제작 때 참고한 제품 구조와의 대응입니다. 둘째 줄, 에이전트는 사람과 같은 사용자 목록의 한 행으로 등록되고, 쓸 도구는 조직 단위로 등록한 MCP 서버에서 나옵니다. 우리 시스템도 같은 방식으로 에이전트를 등록했어요.

셋째 줄부터 프로세스 쪽인데, 이름이 조금 달라요. 흐름과 판본은 프로세스 정의와 정의 버전입니다. 강조한 두 줄을 보세요. 처리 건은 프로세스 인스턴스, 단계는 워크아이템이라고 불러요. 마지막 줄, 사람에게 맡겨진 워크아이템이 모이는 곳이 내 작업함입니다.

이름은 달라도 구실은 같아요. 그러니까 처리 건과 단계를 구별하던 질문을 제품의 인스턴스와 워크아이템에도 적용할 수 있습니다. 상태 이름이 비슷해도 실제 완료 의미는 해당 제품의 정의와 결과 기록을 확인해야 해요.

[SLIDE 52] 실행 엔진, 워크아이템 목록, 수행자

이 대응이 실제로 어떻게 움직이는지, HYD가 참고한 제품의 작업 전달 구조를 한 그림으로 놓았습니다. 이 그림은 실제 제품의 쿨러 실행 화면이 아니라, 배운 사례를 구조에 대응시킨 설명입니다. 왼쪽이 실행 엔진, 가운데가 워크아이템 목록, 오른쪽이 수행자예요.

실행 엔진은 프로세스 정의를 읽고, 수행할 작업을 워크아이템으로 관리하고, 다음 차례의 작업을 진행 상태로 엽니다. 예정 작업을 미리 목록에 만들어 둘 수도 있어요. 오른쪽 수행자는 자기에게 맡겨진 워크아이템을 처리해서 결과를 제출하고요. 엔진은 그 결과를 받아서 다음 워크아이템을 엽니다.

쿨러 사건으로 읽어 볼까요? 목록 맨 위, 진단 · 조치 제안은 에이전트가 끝내서 완료 상태예요. 가운데 운전원 승인은 사람에게 맡겨져서 처리 중이고, 이 워크아이템이 운전원의 작업함에 놓여 있습니다. 맨 아래 다음 작업은 아직 차례가 오지 않은 예정 상태고요.

에이전트가 결과를 제출하면 엔진이 이어받는다는 걸 앞에서 봤죠. 제품에서도 그 구조가 그대로입니다. 에이전트든 사람이든 수행자는 자기 워크아이템만 처리하고, 다음을 여는 건 엔진이에요.

[SLIDE 53] 수업용 구현과 제품 구현의 경계

왼쪽은 HYD가 제품 구조를 참고한 부분입니다. 프로세스 정의와 판본, 실제 인스턴스, 수행자에게 맡기는 작업을 구별했어요. 에이전트가 결과를 제출하고 엔진이 그 결과를 받아 다음 작업을 여는 구조도 참고했습니다. 제품을 그대로 설치해 쓴 것과는 구별해야 해요. 교육용으로 직접 구현한 부분과 적용 범위가 있기 때문입니다.

오른쪽은 제품을 평가할 때 더 살펴볼 범위예요. 자연어로 만든 정의를 어떻게 검토하는지, 여러 에이전트와 하위 프로세스를 어떻게 관리하는지, 지식을 어떤 방식으로 검색하는지, 운영 중에는 권한과 실패를 어떻게 다루는지입니다. 기능이 소개돼 있는 것과 우리 환경에서 그 기능을 검증한 것은 다릅니다.

특히 수업에서 역할을 골라 승인한 경험을 실제 사용자 인증과 혼동하지 마세요. 기업에서는 누가 접속했는지 확인하고, 그 신원에 어떤 권한이 있는지 연결해야 합니다. 같은 승인 상자가 있어도 이 연결이 빠지면 책임을 확인할 수 없죠. 제품이 바뀌었을 때도 살펴봐야 하는 것은 바로 이런 역할과 통제의 실제 연결입니다.

[SLIDE 54] 기업 업무에 적용할 때 필요한 통제 요소

기업 업무에 적용할 때는 표의 여섯 항목을 질문으로 바꿔 보면 좋습니다. 사람 승인은 효과를 내기 전에 필요한 결정이 있었는지, 권한은 그 결정을 할 사람이 맞는지를 묻습니다. 승인 버튼이 보인다는 것만으로 두 질문이 해결되지는 않아요.

감사 추적은 누가 무엇을 결정하고 어떤 실행이 일어났는지 되짚는 길입니다. 제안, 승인, 명령과 결과가 서로 이어져 있어야 원인을 추적할 수 있죠. 이미 한 작업을 다시 시도해도 되는지 판단할 때도 이 기록이 필요합니다.

보상은 실패 뒤에 남은 효과를 정리하는 일입니다. 모든 작업을 과거로 되돌리는 만능 취소가 아니에요. 예를 들어 발주 뒤 다음 단계가 실패했다면, 발주 취소가 가능한지 확인하고 취소 요청이나 정정 절차를 실행해야 합니다. 이미 발송한 안내를 없던 일로 만들 수 없다면 정정 안내가 필요할 수 있고요. BPMN도 보상 작업을 표현할 수 있지만, 무엇을 어떻게 보상할지는 업무별로 설계해야 합니다.

실행 격리는 에이전트와 도구가 접근할 데이터와 기능을 제한합니다. 읽기 도구의 범위, 계정 권한, 연결 가능한 시스템을 함께 봐야 해요. 변경 통제는 어떤 판본으로 실행 중인지와 새 정의를 언제 적용할지를 관리합니다. 처리 건의 판본을 고정해도 센서 값이나 현재 권한까지 고정되는 것은 아닙니다.

오른쪽 칸은 HYD에서 이 원리를 살펴본 자리와 추가로 확인할 지점입니다. 제품을 도입할 때도 각 항목이 실제 설정과 시험에서 어떻게 지켜지는지 확인하세요. 제품 이름이 같다는 이유만으로 우리 업무에 필요한 통제가 모두 갖춰졌다고 결론 내리지 않는 겁니다.

[SLIDE 55] 시연에서 보는 것과 가려낼 것

시연을 시작하기 전에 무엇을 볼지 정리해 두겠습니다. 화면의 카드 네 장이 눈여겨볼 것들이에요. 첫째, 지식과 에이전트와 프로세스가 제품에서 어떤 이름으로 나타나는지. 둘째, 정의 하나에서 열린 인스턴스가 에이전트 작업과 사람 작업으로 어떻게 이어지는지예요.

셋째는 배운 구조 위에 더해진 것들입니다. 말로 만드는 정의나 여러 에이전트의 협업 같은 거요. 넷째는 승인과 권한, 감사 추적 같은 통제가 흐름의 어디에 놓이는지예요.

시연을 본 뒤에는 화면에서 세 가지를 가려낼 수 있어야 합니다. 눈앞의 것이 정의인지 인스턴스인지, 지금이 누구의 차례인지, 사람이 결정하는 자리가 어디인지요. 실제 시연에서는 이 네 가지를 제품 화면에 대어 확인합니다. 준비된 환경에서 확인하지 못한 것은 미확인으로 남기면 됩니다.

[SLIDE 56~57] 실습: 학습 내용과 결과물의 지식 자산화

마지막으로 준비할 실습은 학습 내용과 결과를 개인 지식 자산으로 정리하는 일입니다. 제품 시연까지 마친 뒤에는 정의와 인스턴스, 담당과 결정의 자리를 어디에서 확인했는지도 남길 수 있겠죠. 남은 건 이 모든 걸 내 것으로 묶어 두는 일이에요. 과정을 시작할 때 만든 Notion 학습 노트로 돌아갑니다.

노트에 네 가지를 서로 이어서 기록합니다. 화면의 카드 순서대로, 자신의 문장으로 다시 쓴 개념, 그 개념이 나온 참고 자료, 이번 회차에 만든 명세와 스키마와 에이전트와 흐름 같은 실습 산출물, 그리고 그게 정한 대로 동작했는지의 검증 결과예요.

왜 이렇게 이어 둘까요? 개념만 적힌 노트는 무엇을 알았는지를 보여 줍니다. 그런데 산출물과 검증 결과가 이어진 노트는 무엇을 할 수 있는지를 보여 줘요. 수업 계정에서 Notion 연결과 쓰기 권한이 확인되면 MCP로 연동해서 기록하는 모습을 보여 드리고요, 그렇지 않은 환경에서는 준비한 Markdown을 Notion에 직접 정리합니다. 개념에서 출처와 산출물로, 산출물에서 검증 결과로 끊기지 않고 이어지면 이 노트는 완성입니다. 이제 실제 실습과 시연에서는 안내한 순서대로 결과를 확인하고, 마지막에 네 가지를 이어 기록하겠습니다. 마치고 돌아오면 복습과 퀴즈로 과정을 정리합니다.

[SLIDE 58] 복습: 쿨러 사건의 순간을 맡은 영역으로 가르기

실습과 시연을 마친 뒤 다시 모였습니다. 이제 복습과 용어 정리, 퀴즈로 이번 회차를 마무리하겠습니다. 학습 노트에는 완성한 결과뿐 아니라 아직 확인하지 못한 것도 구분해 두세요. 먼저 이번 회차에 배운 흐름을 쿨러 사건으로 복습해 보겠습니다. 표에 쿨러 사건의 다섯 순간을 놓았어요. 답과 근거 칸은 잠깐 가려 둘게요. 순간마다 지식, 에이전트, 프로세스 가운데 어느 영역의 일인지 먼저 골라 보세요. 왜 그 영역인지 근거도 같이 생각해 보시고요.

답을 맞춰 볼까요? 첫째, 핀 오염의 기준을 읽은 순간은 지식의 일입니다. 여기서 지식으로 분류한 것은 읽는 행동이 아니라 기준의 출처와 역할입니다. 기준은 매뉴얼과 정책을 검토해 관리하고 개정하는 지식이죠. 둘째, 지금의 냉각 효율을 읽어서 기준에 댄 건 에이전트의 일이에요. 값은 도구로 읽은 현재 값이고, 그걸 기준에 대어 판단하는 게 에이전트의 몫이니까요.

셋째, 쿨러 핀 세척을 설비보전팀장이 승인한다는 건 다시 지식입니다. 승인 역할은 조치에 적힌 지식이었죠. 넷째, 운전원 승인 뒤에 냉각 명령이 열린 건 프로세스의 일이에요. 단계 사이의 순서는 엔진이 지킵니다.

다섯째가 헷갈리기 쉬워요. 내려간 유온을 다시 읽고 작업지시를 등록한 건 프로세스의 일입니다. 둘째 순간처럼 시계열 DB를 읽긴 하는데, 판단의 재료를 읽은 게 아니라 승인 뒤 실행의 결과를 확인한 서비스 작업이거든요.

결국 같은 값을 읽어도, 누가 무엇을 위해 읽었는지에 따라 영역이 갈립니다. 이 HYD 사례에서는 진단 도구를 통한 대조와 승인 뒤 재관측 서비스의 역할이 다릅니다. 다른 시스템에서는 배치가

달라질 수 있어도, 판단 근거를 읽는 일과 실행 결과를 확인하는 일을 구별하는 질문은 유효합니다.

[SLIDE 59] 오늘 다룬 핵심 개념과 주요 용어 정리

흐름을 되짚었으니, 이번엔 용어의 이름과 뜻을 정리하겠습니다. 표에 이번 회차에서 다시 쓰고 새로 세운 용어 일곱 개가 있어요.

스킬은 에이전트가 업무를 수행하는 순서와 요령을 적은 재사용 지침입니다. MCP는 에이전트가 외부 시스템의 도구를 같은 방식으로 부르게 하는 연결 규약이에요. 출력 계약은 에이전트의 결과에 반드시 있어야 할 항목과 형식의 약속입니다. HITL은 사람의 판단이나 피드백을 과정 안에 넣는 방식입니다. 이 수업에서는 실행 전에 승인하거나 거절하는 자리로 구체화했고요.

프로세스 인스턴스는 프로세스 정의에 따라 실제로 도는 한 건이에요. 우리가 처리 건이라고 불러온 거죠. 워크아이템은 인스턴스 안에서 수행자에게 맡겨지는 작업 하나, 우리가 단계라고 부른 겁니다. 마지막으로 멀티 에이전트 오케스트레이션은 한 프로세스 안에서 여러 에이전트의 순서와 호출을 조율하는 일입니다.

[SLIDE 60] Review Quiz: 핵심 흐름 되짚어보기

배운 내용을 판단에 쓸 수 있는지, 퀴즈 세 문제로 확인해 보겠습니다. 보기의 말을 기억하는 것보다, 그 판단이 어느 영역의 몫인지 근거를 댈 수 있는지가 중요해요. 문제마다 보기를 읽고 나면 잠깐 멈출 테니, 그때 답을 골라 보세요. 정답과 까닭은 세 문제를 다 푼 뒤에 함께 확인하겠습니다.

[SLIDE 61] Quiz 1. 추천안이 제출된 시점의 처리 건

첫 번째 문제입니다. 상황부터 볼게요. 에이전트가 원인을 쿨러 핀 오염으로 가리고, 팬 최대 운전을 권하는 추천안을 제출했습니다. 운전원은 아직 응답하지 않았어요. 이 시점의 처리 건에 대한 설명으로 옳은 건 무엇일까요?

1번, 추천안이 제출됐으니 팬 속도는 이미 100퍼센트로 올라갔다. 2번, 에이전트가 승인 결과를 기다리며 다음 도구를 부르고 있다. 3번, 에이전트의 일은 끝났고 엔진이 운전원 승인을 열어 기다린다. 4번, 운전원이 응답하지 않으면 추천대로 자동 실행된다.

[SLIDE 62] Quiz 2. 바뀐 기준을 반영할 곳

두 번째 문제입니다. 매뉴얼이 개정돼서, 쿨러 핀 오염을 가리는 냉각 효율의 기준값이 달라졌어요. 그런데 에이전트는 여전히 예전 기준으로 판단합니다. 조회 구조와 기준의 단위는 그대로라고 할 때, 새 기준이 판단에 쓰이게 하려면, 먼저 고칠 곳은 어디일까요?

1번, 스킬의 절차에 새 기준값을 문장으로 적어 넣는다. 2번, 개정된 문서를 검토해 증거 기준을 다시 적재한다. 3번, 프로세스 정의의 게이트웨이 조건을 새 기준값으로 바꾼다. 4번, 운전원이 승인할 때마다 새 기준값을 확인하게 한다.

[SLIDE 63] Quiz 3. 발주를 직접 내는 도구를 연결하자는 의견

마지막 문제입니다. 재고 보충을 빨리 끝내려고, 구매 에이전트에게 발주를 직접 등록하는 쓰기 도구를 연결하자는 의견이 나왔어요. 통제의 관점에서 가장 알맞은 판단은 무엇일까요?

1번, 발주 등록은 승인 뒤의 프로세스 작업에 두고 에이전트는 읽기만 한다. 2번, 스킬에 승인 뒤에만 발주한다고 적어 두면 쓰기 도구를 연결해도 된다. 3번, 발주한 뒤에 감사 기록이 남으니 쓰기 도구를 연결해도 문제없다. 4번, 발주 수량을 규칙으로 계산하게 하면 사람의 승인은 필요 없다.

[SLIDE 64] 정답과 판단 기준

정답을 확인해 보겠습니다. 고른 답과 근거를 같이 대조해 보세요. 첫 문항의 정답은 3번이에요. 에이전트의 일은 추천안을 제출하는 데서 끝나고, 운전원 승인 단계를 여는 건 엔진입니다. 설비 명령은 승인이 전달된 뒤에만 나가니까, 이 시점에 팬은 아직 그대로예요.

1번은 제출과 실행을 같은 걸로 본 보기예요. 2번은 에이전트가 승인을 기다린다고 본 건데, 에이전트는 다음에 누가 무엇을 하는지 몰라도 됩니다. 4번은 응답 없음을 승인으로 읽은 거예요. 응답이 없으면 지연 알림만 나가고, 처리 건은 계속 기다리죠.

둘째 문항의 정답은 2번입니다. 증거 기준은 매뉴얼에서 뽑아 적재한 지식이고, 진단 도구가 판단할 때마다 그 기준을 읽어요. 이 문제처럼 조회 구조와 기준의 단위가 유지된다면, 개정된 문서를 검토해 기준을 다시 적재하고 판단에 반영됐는지 확인합니다. 1번처럼 스킬에 적으면 기준이 두 곳에 생겨서, 문서가 다시 바뀔 때 어긋나요. 3번의 게이트웨이는 승인과 회복을 가르는 조건이라 원인의 기준과는 관계가 없습니다. 4번은 기준을 사람의 기억에 맡기는 셈이고요.

셋째 문항의 정답은 1번입니다. 업무 시스템에 쓰는 일은 승인 뒤의 프로세스 작업에 두고, 이 사례의 에이전트에는 읽기와 제안 제출에 필요한 권한을 둡니다. 2번은 스킬에 적은 순서를 엔진이 지키는 순서와 헷갈린 보기예요. 지침은 따르는 쪽에 맡겨질 뿐, 보장되지 않죠. 3번의 감사 기록은 일이 벌어진 뒤에 남는 거라서 승인을 대신하지 못하고요, 4번은 계산이 아무리 정확해도 책임질 사람의 결정이 빠집니다.

여기까지가 이번 회차입니다.

그리고 이 회차로 과정 전체를 마칩니다. 앞으로 다른 제품이나 도구를 만나더라도, 이 과정에서 익힌 원리와 역할의 구분은 그대로 쓸 수 있을 거예요. 이번 회차에 정리한 학습 노트, 그리고 직접 만든 명세와 에이전트와 흐름이 각자의 업무에서 다음 일을 시작하는 출발점이 되면 좋겠습니다.
그동안 수고 많으셨습니다. 감사합니다.